



ZAP
by
Checkmarx

ZAP Authenticated Active Scan Report - Backend API

main @ 4325315a, authenticated (admin) active scan against cloned DB

Site: <http://localhost:8100>

Generated on Thu, 6 Aug 2026 03:36:10

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	2
Medium	1
Low	4
Informational	2
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Path Traversal	High	2
Remote OS Command Injection	High	5
Format String Error	Medium	7
Application Error Disclosure	Low	5
Cross Site Scripting Weakness (Persistent in JSON Response)	Low	11
Information Disclosure - Debug Error Messages	Low	3
X-Content-Type-Options Header Missing	Low	Systemic
Information Disclosure - Sensitive Information in URL	Informational	5
User Agent Fuzzer	Informational	Systemic

Alert Detail

High	Path Traversal
Description	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters ("%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.
URL	http://localhost:8100/api/v1/auth/dev-profiles/quick-setup/quick-setup
Node Name	http://localhost:8100/api/v1/auth/dev-profiles/quick-setup/quick-setup
Method	POST
Parameter	<<developer_id>>
Attack	quick-setup
Evidence	
Other Info	
URL	http://localhost:8100/api/v1/auth/dev-profiles/staff-suite/staff-suite
Node Name	http://localhost:8100/api/v1/auth/dev-profiles/staff-suite/staff-suite
Method	POST
Parameter	<<developer_id>>
Attack	staff-suite
Evidence	
Other Info	
Instances	2
Solution	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists

	can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions. Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised. Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked. Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes "." sequences and symbolic links. Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations. When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.
Reference	https://owasp.org/www-community/attacks/Path_Traversal https://cwe.mitre.org/data/definitions/22.html
CWE Id	22
WASC Id	33
Plugin Id	6
High	Remote OS Command Injection
Description	Attack technique used for unauthorized execution of operating system commands. This attack is possible when an application accepts untrusted input to build operating system commands in an insecure manner involving improper data sanitization, and/or improper calling of external programs.
URL	http://localhost:8100/api/v1/admin/professors?search=get-help

Node Name	http://localhost:8100/api/v1/admin/professors (search)
Method	GET
Parameter	search
Attack	get-help
Evidence	Get-Help
Other Info	The scan rule was able to retrieve the content of a file or command by sending [get-help] to the operating system running this application.
URL	http://localhost:8100/api/v1/auth/dev-profiles/get-help
Node Name	http://localhost:8100/api/v1/auth/dev-profiles/get-help
Method	GET
Parameter	«developer_id»
Attack	get-help
Evidence	Get-Help
Other Info	The scan rule was able to retrieve the content of a file or command by sending [get-help] to the operating system running this application.
URL	http://localhost:8100/api/v1/users?page=1&size=20&role=&roles=&search=get-help&include_permissions=false
Node Name	http://localhost:8100/api/v1/users (include_permissions,page,role,roles,search,size)
Method	GET
Parameter	search
Attack	get-help
Evidence	Get-Help
Other Info	The scan rule was able to retrieve the content of a file or command by sending [get-help] to the operating system running this application.
URL	http://localhost:8100/api/v1/auth/dev-profiles/get-help/quick-setup
Node Name	http://localhost:8100/api/v1/auth/dev-profiles/get-help/quick-setup
Method	POST
Parameter	«developer_id»
Attack	get-help
Evidence	Get-Help
Other Info	The scan rule was able to retrieve the content of a file or command by sending [get-help] to the operating system running this application.
URL	http://localhost:8100/api/v1/auth/dev-profiles/get-help/staff-suite
Node Name	http://localhost:8100/api/v1/auth/dev-profiles/get-help/staff-suite
Method	POST
Parameter	«developer_id»
Attack	get-help
Evidence	Get-Help

Other Info	The scan rule was able to retrieve the content of a file or command by sending [get-help] to the operating system running this application.
Instances	5
Solution	If at all possible, use library calls rather than external processes to recreate the desired functionality. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise. For any data that will be used to generate a command to be executed, keep as much of that data out of external control as possible. For example, in web applications, this may require storing the command locally in the session's state instead of sending it out to the client in a hidden form field. Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, consider using the ESAPI Encoding control or a similar tool, library, or framework. These will help the programmer encode outputs in a manner less prone to error. If you need to use dynamically-generated query strings or commands in spite of the risk, properly quote arguments and escape any special characters within those arguments. The most conservative approach is to escape or filter all characters that do not pass an extremely strict allow list (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection. If the program to be executed allows arguments to be specified within an input file or from standard input, then consider using that mode to pass arguments instead of the command line. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Some languages offer multiple functions that can be used to invoke commands. Where possible, identify any function that invokes a command shell using a single string, and replace it with a function that requires individual arguments. These functions typically perform appropriate quoting and filtering of arguments. For example, in C, the system() function accepts a string that contains the entire command to be executed, whereas execl(), execve(), and others require an array of strings, one for each argument. In Windows, CreateProcess() only accepts one command at a time. In Perl, if system() is provided with an array of arguments, then it will quote each of the arguments. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

	When constructing OS command strings, use stringent allow lists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping. Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components. Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.
Reference	https://cwe.mitre.org/data/definitions/78.html https://owasp.org/www-community/attacks/Command_Injection
CWE Id	78
WASC Id	31
Plugin Id	90020

Medium	Format String Error
Description	A Format String error occurs when the submitted data of an input string is evaluated as a command by the application.
URL	http://localhost:8100/api/v1/application-fields/fields
Node Name	http://localhost:8100/api/v1/application-fields/fields () ({scholarship_type,field_name,field_label,field_label_en,field_type,is_required,placeholder,placeholder_en,max_length,min_value,max_value,step_value,field_options: [{},display_order,is_active,help_text,help_text_en,validation_rules:{},conditional_rules:{},include_in_college_export,export_column_label])
Method	POST
Parameter	field_name
Attack	ZAP %1!s%2!s%3!s%4!s%5!s%6!s%7!s%8!s%9!s%10!s%11!s%12!s%13!s%14!s%15!s%16!s%17!s%18!s%19!s%20!s%21!n%22!n%23!n%24!n%25!n%26!n%27!n%28!n%29!n%30!n%31!n%32!n%33!n%34!n%35!n%36!n%37!n%38!n%39!n%40!n
Evidence	
Other Info	Potential Format String Error. The script closed the connection on a Microsoft format string error.
URL	http://localhost:8100/api/v1/application-fields/fields
Node Name	http://localhost:8100/api/v1/application-fields/fields () ({scholarship_type,field_name,field_label,field_label_en,field_type,is_required,placeholder,placeholder_en,max_length,min_value,max_value,step_value,field_options: [{},display_order,is_active,help_text,help_text_en,validation_rules:{},conditional_rules:{},include_in_college_export,export_column_label])
Method	POST
Parameter	scholarship_type
Attack	ZAP%x%x%x%x%x%x%x%x%x%x

[illegible]

Node Name	http://localhost:8100/api/v1/admin/system-setting (){key,value}}
Method	PUT
Parameter	key
Attack	ZAP %1!s%2!s%3!s%4!s%5!s%6!s%7!s%8!s%9!s%10!s%11!s%12!s%13!s%14!s%15!s%16!s%17!s%18!s%19!s%20!s%21!n%22!n%23!n%24!n%25!n%26!n%27!n%28!n%29!n%30!n%31!n%32!n%33!n%34!n%35!n%36!n%37!n%38!n%39!n%40!n
Evidence	
Other Info	Potential Format String Error. The script closed the connection on a Microsoft format string error.
Instances	7
Solution	Rewrite the background program using proper deletion of bad character strings. This will require a recompile of the background executable.
Reference	https://owasp.org/www-community/attacks/Format_string_attack
CWE Id	134
WASC Id	6
Plugin Id	30002

Low	Application Error Disclosure
Description	This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.
URL	http://localhost:8100/api/v1/manual-distribution/quota-status?scholarship_type_id=10&academic_year=10&semester=semester
Node Name	http://localhost:8100/api/v1/manual-distribution/quota-status (academic_year,scholarship_type_id,semester)
Method	GET
Parameter	
Attack	
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
URL	http://localhost:8100/api/v1/manual-distribution/allocate
Node Name	http://localhost:8100/api/v1/manual-distribution/allocate () ({scholarship_type_id,academic_year,semester,allocations: [{ranking_item_id,sub_type_code,allocation_config_id}]})
Method	POST
Parameter	
Attack	
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
URL	http://localhost:8100/api/v1/notifications/admin/create-test-notifications
Node Name	http://localhost:8100/api/v1/notifications/admin/create-test-notifications
Method	POST

Parameter	
Attack	
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
URL	http://localhost:8100/api/v1/payment-rosters/10/dry-run
Node Name	http://localhost:8100/api/v1/payment-rosters/10/dry-run () ({scholarship_configuration_id,period_label,roster_cycle,academic_year,student_verification_enabled,ranking_id,auto_export_excel,force_regenerate})
Method	POST
Parameter	
Attack	
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
URL	http://localhost:8100/api/v1/scholarship-configurations/matrix-quota
Node Name	http://localhost:8100/api/v1/scholarship-configurations/matrix-quota () ({sub_type,college,new_quota,academic_year})
Method	PUT
Parameter	
Attack	
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
Instances	5
Solution	Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server side and not exposing them to the user.
Reference	
CWE Id	550
WASC Id	13
Plugin Id	90022

Low	Cross Site Scripting Weakness (Persistent in JSON Response)
Description	A XSS attack was found in a JSON response, this might leave content consumers vulnerable to attack if they don't appropriately handle the data (response).
URL	http://localhost:8100/api/v1/admin/announcements
Node Name	http://localhost:8100/api/v1/admin/announcements
Method	GET
Parameter	message
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements

Node Name	http://localhost:8100/api/v1/admin/announcements
Method	GET
Parameter	message_en
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements
Node Name	http://localhost:8100/api/v1/admin/announcements
Method	GET
Parameter	title
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements
Node Name	http://localhost:8100/api/v1/admin/announcements
Method	GET
Parameter	title_en
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements?page=1&size=20&notification_type=&priority=
Node Name	http://localhost:8100/api/v1/admin/announcements (notification_type,page,priority,size)
Method	GET
Parameter	action_url
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements?page=1&size=20&notification_type=&priority=
Node Name	http://localhost:8100/api/v1/admin/announcements (notification_type,page,priority,size)
Method	GET
Parameter	message
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements?page=1&size=20&notification_type=&priority=

Node Name	http://localhost:8100/api/v1/admin/announcements (notification_type,page,priority,size)
Method	GET
Parameter	message_en
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements?page=1&size=20&notification_type=&priority=
Node Name	http://localhost:8100/api/v1/admin/announcements (notification_type,page,priority,size)
Method	GET
Parameter	title
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/admin/announcements?page=1&size=20&notification_type=&priority=
Node Name	http://localhost:8100/api/v1/admin/announcements (notification_type,page,priority,size)
Method	GET
Parameter	title_en
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/footer-links
Node Name	http://localhost:8100/api/v1/footer-links
Method	GET
Parameter	title_en
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
URL	http://localhost:8100/api/v1/footer-links
Node Name	http://localhost:8100/api/v1/footer-links
Method	GET
Parameter	title_zh
Attack	<script>alert(1);</script>
Evidence	
Other Info	Raised with LOW confidence as the Content-Type is not HTML.
Instances	11
Solution	Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.

Phases: Implementation; Architecture and Design

Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies.

For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters.

Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed.

Phase: Architecture and Design

For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Phase: Implementation

For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.

To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Reference	https://owasp.org/www-community/attacks/xss/ https://cwe.mitre.org/data/definitions/79.html
CWE Id	79
WASC Id	8
Plugin Id	40014

Low	Information Disclosure - Debug Error Messages
Description	The response appeared to contain common error messages returned by platforms such as ASP.NET, and Web-servers such as IIS and Apache. You can configure the list of common debug messages.
URL	http://localhost:8100/api/v1/manual-distribution/quota-status?scholarship_type_id=10&academic_year=10&semester=semester
Node Name	http://localhost:8100/api/v1/manual-distribution/quota-status (academic_year,scholarship_type_id,semester)
Method	GET
Parameter	
Attack	
Evidence	Internal server error
Other Info	
URL	http://localhost:8100/api/v1/manual-distribution/allocate
Node Name	http://localhost:8100/api/v1/manual-distribution/allocate () ({scholarship_type_id,academic_year,semester,allocations: [{ranking_item_id,sub_type_code,allocation_config_id}]})
Method	POST
Parameter	
Attack	
Evidence	Internal server error
Other Info	
URL	http://localhost:8100/api/v1/notifications/admin/create-test-notifications
Node Name	http://localhost:8100/api/v1/notifications/admin/create-test-notifications
Method	POST
Parameter	
Attack	
Evidence	Internal server error
Other Info	
Instances	3
Solution	Disable debugging messages before pushing to production.
Reference	
CWE Id	1295
WASC Id	13
Plugin Id	10023

Low	X-Content-Type-Options Header Missing
------------	--

Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://localhost:8100/
Node Name	http://localhost:8100/
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8100/api/v1/auth/me
Node Name	http://localhost:8100/api/v1/auth/me
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8100/debug/pool-status
Node Name	http://localhost:8100/debug/pool-status
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8100/health
Node Name	http://localhost:8100/health
Method	GET
Parameter	x-content-type-options
Attack	

Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8100/api/v1/auth/refresh
Node Name	http://localhost:8100/api/v1/auth/refresh
Method	POST
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Information Disclosure - Sensitive Information in URL
Description	The request appeared to contain sensitive information leaked in the URL. This can violate PCI and most organizational compliance policies. You can configure the list of strings for this check to add or remove values specific to your environment.
URL	http://localhost:8100/api/v1/admin/audit-logs?page=1&size=50&resource_type=&resource_id=&action=&user_id=&date_from=&date_to=&search=ZAP
Node Name	http://localhost:8100/api/v1/admin/audit-logs (action,date_from,date_to,page,resource_id,resource_type,search,size,user_id)
Method	GET
Parameter	user_id
Attack	
Evidence	user_id
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user user_id
URL	http://localhost:8100/api/v1/admin/scholarship-permissions?user_id=

Node Name	http://localhost:8100/api/v1/admin/scholarship-permissions (user_id)
Method	GET
Parameter	user_id
Attack	
Evidence	user_id
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user user_id
URL	http://localhost:8100/api/v1/files/applications/10/files/10?token=
Node Name	http://localhost:8100/api/v1/files/applications/10/files/10 (token)
Method	GET
Parameter	token
Attack	
Evidence	token
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: token token
URL	http://localhost:8100/api/v1/files/applications/10/files/10/download?token=
Node Name	http://localhost:8100/api/v1/files/applications/10/files/10/download (token)
Method	GET
Parameter	token
Attack	
Evidence	token
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: token token
URL	http://localhost:8100/api/v1/user-profiles/files/bank_documents/filename?token=
Node Name	http://localhost:8100/api/v1/user-profiles/files/bank_documents/filename (token)
Method	GET
Parameter	token
Attack	
Evidence	token
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: token token
Instances	5
Solution	Do not pass sensitive information in URIs.
Reference	

CWE Id	598
WASC Id	13
Plugin Id	10024

Informational	User Agent Fuzzer
Description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
URL	http://localhost:8100/api/v1/admin/announcements/10
Node Name	http://localhost:8100/api/v1/admin/announcements/10
Method	DELETE
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://localhost:8100/api/v1/admin/announcements/10
Node Name	http://localhost:8100/api/v1/admin/announcements/10 ()({"title":"John Doe","title_en":"John Doe","message":"Zaproxy alias impedit expedita quisquam pariatur exercitationem. Nemo rerum eveniet dolores rem quia dignissimos.,"message_en":"John Doe","notification_type":"John Doe","priority":"John Doe","action_url":"John Doe","expires_at":"1970-01-01T00:00:00.001Z","metadata":{"is_dismissed":true}})
Method	PUT
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
Evidence	
Other Info	
URL	http://localhost:8100/api/v1/admin/announcements/10
Node Name	http://localhost:8100/api/v1/admin/announcements/10 ()({"title":"John Doe","title_en":"John Doe","message":"Zaproxy alias impedit expedita quisquam pariatur exercitationem. Nemo rerum eveniet dolores rem quia dignissimos.,"message_en":"John Doe","notification_type":"John Doe","priority":"John Doe","action_url":"John Doe","expires_at":"1970-01-01T00:00:00.001Z","metadata":{"is_dismissed":true}})
Method	PUT
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
Instances	Systemic
Solution	
Reference	https://owasp.org/wstg
CWE Id	
WASC Id	
Plugin Id	10104

Sequence Details

With the associated active scan results.